

Predicting Major US Power Outage Severity

Name: Zack Mosley

Website Link: https://zmosds.github.io/predict_severe_power_outages/

```
In [3]: import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingC
from sklearn.metrics import classification_report, average_precision_s
from lightgbm import LGBMClassifier
import pickle
import plotly.figure_factory as ff
import plotly.express as px
import plotly.io as pio

pd.options.plotting.backend = 'plotly'
pio.renderers.default = 'notebook'

import warnings
warnings.filterwarnings('ignore')

# from dsc259r_utils import * # Feel free to uncomment and use this.
```

```
In [4]: # Load data and select relevant columns
cols = ['YEAR', 'MONTH', 'U.S._STATE', 'POSTAL.CODE', 'NERC.REGION', '
        'CLIMATE.CATEGORY', 'ANOMALY.LEVEL', 'CAUSE.CATEGORY', 'CAUSE.
        'OUTAGE.START.DATE', 'OUTAGE.START.TIME', 'OUTAGE.RESTORATION.
        'OUTAGE.DURATION', 'CUSTOMERS.AFFECTED', 'DEMAND.LOSS.MW',
        'TOTAL.CUSTOMERS', 'POPDEN_URBAN', 'PC.REALGSP.STATE', 'UTIL.C
        'POPPCT_URBAN', 'PC.REALGSP.REL', 'RES.CUST.PCT', 'TOTAL.REALG

df = pd.read_excel('../data/power_outages_us.xlsx', header=5, usecols=
df = df.reset_index(drop=True)
print("Data loaded.\n")
df.head(1)
```

Data loaded.

```
Out[4]:
```

	YEAR	MONTH	U.S._STATE	POSTAL.CODE	NERC.REGION	CLIMATE.REGION
0	2011	7.0	Minnesota	MN	MRO	East North Central

1 rows x 25 columns

Step 1: Introduction

Question:

What factors best predict how long a power outage will last?

Introduction

Power outages affect millions of Americans every year, disrupting daily life, businesses, and critical infrastructure. Understanding what drives outage duration could help utilities better prepare for and respond to future events. This project uses a dataset of major power outages in the United States from 2000 to 2016, containing 1534 rows and covering outage causes, climate conditions, geographic information, and economic data.

Column	Description
OUTAGE.DURATION	Duration of the outage in minutes (prediction target)
CAUSE.CATEGORY	High level category of the outage cause (e.g. severe weather, intentional attack)
CLIMATE.CATEGORY	Climate episode category during the outage (warm, cold, or normal)
U.S._STATE	State where the outage occurred
NERC.REGION	North American Electric Reliability Corporation region
ANOMALY.LEVEL	Oceanic El Nino/La Nina index indicating climate anomaly severity
CUSTOMERS.AFFECTED	Number of customers affected by the outage
DEMAND.LOSS.MW	Amount of peak demand lost in megawatts

Step 2: Data Cleaning and Exploratory Data Analysis

In [5]: *# Cleaning pipeline*

```
def drop_missing_start(df):
    # Drop rows missing start date/time -- can't construct timestamp
    df = df[df['OUTAGE.START.DATE'].notna() & df['OUTAGE.START.TIME'].notna()]
    return df

def combine_timestamps(df):
    # Combine start date and time into single timestamp
    df['OUTAGE.START'] = pd.to_datetime(
        df['OUTAGE.START.DATE'].astype(str) + ' ' + df['OUTAGE.START.TIME'].astype(str),
        format='mixed', errors='coerce')

    # Combine restoration date and time into single timestamp
    df['OUTAGE.RESTORATION'] = pd.to_datetime(
        df['OUTAGE.RESTORATION.DATE'].astype(str) + ' ' + df['OUTAGE.RESTORATION.TIME'].astype(str),
        format='mixed', errors='coerce')

    # Drop the original separate columns
    df = df.drop(columns=['OUTAGE.START.DATE', 'OUTAGE.START.TIME', 'OUTAGE.RESTORATION.DATE', 'OUTAGE.RESTORATION.TIME'])
    return df

def drop_invalid_durations(df):
    # Drop rows where duration is missing or zero -- can't predict or
    df = df[df['OUTAGE.DURATION'].notna() & (df['OUTAGE.DURATION'] > 0)]
    return df

def drop_non_continental(df):
    # Hawaii and Alaska aren't part of the continental U.S. climate re
    df = df[~df['U.S._STATE'].isin(['Hawaii', 'Alaska'])]
    return df
```

In [6]: *# Apply cleaning pipeline*

```
df = (df
      .pipe(drop_missing_start)
      .pipe(drop_invalid_durations)
      .pipe(drop_non_continental)
      .pipe(combine_timestamps)
      )
print("Applied cleaning pipeline.\n")
print(df.shape)
```

Applied cleaning pipeline.

(1393, 23)

In [7]: *# Print head as markdown table for website*

```
cols_to_show = ['YEAR', 'MONTH', 'U.S._STATE', 'CAUSE.CATEGORY', 'CLIMATE.CATEGORY',
                'OUTAGE.DURATION', 'CUSTOMERS.AFFECTED', 'OUTAGE.START DATE', 'OUTAGE.RESTORATION DATE']
print(df[cols_to_show].head().fillna('-').to_markdown(index=False))
```

YEAR	MONTH	U.S._STATE	CAUSE.CATEGORY	CLIMATE.CATEGORY
OUTAGE.DURATION	CUSTOMERS.AFFECTED	OUTAGE.START		
2011	7	Minnesota	severe weather	normal
3060	70000.0		2011-07-01 17:00:00-01:00	
2014	5	Minnesota	intentional attack	normal
1	-		2014-05-11 18:38:00-01:00	
2010	10	Minnesota	severe weather	cold
3000	70000.0		2010-10-26 20:00:00-01:00	
2012	6	Minnesota	severe weather	normal
2550	68200.0		2012-06-19 04:30:00-01:00	
2015	7	Minnesota	severe weather	warm
1740	250000.0		2015-07-18 02:00:00-01:00	

Univariate Analysis

```
In [8]: def plot_univariate(df):
# Distribution of outage duration (target variable)
uni_fig1 = px.histogram(df, x='OUTAGE.DURATION',
                        title='Distribution of Outage Duration',
                        labels={'OUTAGE.DURATION': 'Outage Duration'})

uni_fig1.update_xaxes(range=[-1, 20000]) # Trim tail for better readability
uni_fig1.update_yaxes(title_text='# of Outages')

uni_fig1.write_image('../figures/univariate/outage_duration_dist.png')
uni_fig1.write_html('../figures/univariate/outage_duration_dist.html')
uni_fig1.show()

# Count of outages by cause
cause_counts = df['CAUSE.CATEGORY'].value_counts().reset_index()
cause_counts.columns = ['Cause', 'Count']

uni_fig2 = px.bar(cause_counts, x='Cause', y='Count',
                  title='Power Outages by Cause',
                  labels={'Cause': 'Cause', 'Count': '# of Outages'})

uni_fig2.update_layout(xaxis_tickangle=-35, showlegend=False)
uni_fig2.write_image('../figures/univariate/outage_cause_counts.png')
uni_fig2.show()

# Distribution of anomaly level
uni_fig3 = px.histogram(df, x='ANOMALY.LEVEL',
                        title='Distribution of El Niño and La Niña')
```

```
labels={'ANOMALY.LEVEL': 'El Niño / La Niña'}

uni_fig3.update_xaxes(tickvals=[-1, 0, 1.5],
                      ticktext=["La Niña (Cooler)", "Normal", "El Niño (Warmer)"])
uni_fig3.update_yaxes(title_text='# of Outages')

uni_fig3.write_image('../figures/univariate/anomaly_level_dist.png')
uni_fig3.show()

plot_univariate(df)
```



```
In [9]: def plot_bivariate(df):

    # Average outage duration by cause
    cause_duration = df.groupby('CAUSE.CATEGORY')['OUTAGE.DURATION'].m
    cause_duration.columns = ['Cause', 'Mean Duration']
    cause_duration['Mean Duration'] = cause_duration['Mean Duration']
    cause_duration = cause_duration.sort_values('Mean Duration', ascen

    bi_fig1 = px.bar(cause_duration, x='Cause', y='Mean Duration', tit
        labels={'Cause': 'Cause Category', 'Mean Duration': 'Average D
    )

    bi_fig1.update_layout(xaxis_tickangle=-35)
    bi_fig1.write_image('../figures/bivariate/duration_by_cause.png')
    bi_fig1.write_html('../figures/bivariate/duration_by_cause.html',
    bi_fig1.show()

    # Outage duration vs customers affected (in hours)
    bi_fig2 = px.scatter(
```

```
df,
x='CUSTOMERS.AFFECTED',
y=df['OUTAGE.DURATION'] / 60,
title='Outage Duration vs Customers Affected',
labels={
    'CUSTOMERS.AFFECTED': 'Customers Affected',
    'y': 'Outage Duration (hours)'
}
)

bi_fig2.write_image('../figures/bivariate/duration_vs_customers.png')
bi_fig2.show()

# Outage duration by climate category (in hours)
bi_fig3 = px.box(
    df,
    x='CLIMATE.CATEGORY',
    y=df['OUTAGE.DURATION'] / 60,
    title='Outage Duration by Climate Category',
    labels={
        'CLIMATE.CATEGORY': 'Climate Category',
        'y': 'Outage Duration (hours)'
    },
    category_orders={'CLIMATE.CATEGORY': ['cold', 'normal', 'warm']
}

bi_fig3.write_image('../figures/bivariate/duration_by_climate.png')
bi_fig3.show()

plot_bivariate(df)
```



```
In [10]: def plot_aggregation(df):  
  
    # Mean duration by Cause and Climate Category (hours)  
    cause_climate = (  
        df.groupby(['CAUSE.CATEGORY', 'CLIMATE.CATEGORY'])['OUTAGE.DURATION']  
        .mean()  
        .reset_index()  
    )  
  
    cause_climate['Mean Duration'] = cause_climate['OUTAGE.DURATION']  
    cause_climate = cause_climate.drop(columns='OUTAGE.DURATION')  
  
    agg_fig1 = px.bar(  
        cause_climate,  
        x='CAUSE.CATEGORY',  
        y='Mean Duration',  
        color='CLIMATE.CATEGORY',  
        title='Mean Outage Duration by Cause and Climate Category',  
        labels={  
            'CAUSE.CATEGORY': 'Cause Category',
```

```
        'CLIMATE.CATEGORY': 'Climate Category',
        'Mean Duration': 'Average Duration (hours)'
    },
    barmode='group'
)

agg_fig1.update_layout(xaxis_tickangle=-35)
agg_fig1.show()

# Pivot table: mean outage duration (hours) by Cause x Climate Cat
pivot = (
    df.groupby(['CAUSE.CATEGORY', 'CLIMATE.CATEGORY'])['OUTAGE.DUR']
      .mean()
      .unstack()
      / 60
).round(1)

pivot.index.name = 'Cause Category'
pivot.columns.name = None

pivot.to_html('../figures/aggregation/cause_climate_pivot.html', f

display(pivot)

plot_aggregation(df)
```

	cold	normal	warm
Cause Category			
equipment failure	5.5	53.4	8.4
fuel supply emergency	290.6	127.6	380.0
intentional attack	11.8	8.4	5.3
islanding	4.3	2.4	3.5
public appeal	35.4	22.9	9.9
severe weather	55.0	68.2	74.3
system operability disruption	10.6	15.9	8.2

Step 3: Assessment of Missingness

```
In [11]: # Binary indicator: True if CUSTOMERS.AFFECTED is missing
```

```

df['CUSTOMERS_MISSING'] = df['CUSTOMERS.AFFECTED'].isna()

def permutation_test(df, group_col, missing_col, n_perm=1000):
    # Test statistic: max minus min missingness rate across groups
    observed = (
        df.groupby(group_col)[missing_col]
        .mean()
        .max() -
        df.groupby(group_col)[missing_col]
        .mean()
        .min()
    )

    perm_stats = []

    for _ in range(n_perm):
        shuffled = df.copy()
        shuffled[missing_col] = np.random.permutation(shuffled[missing_col])

        stat = (
            shuffled.groupby(group_col)[missing_col]
            .mean()
            .max() -
            shuffled.groupby(group_col)[missing_col]
            .mean()
            .min()
        )

        perm_stats.append(stat)

    p_value = np.mean(np.array(perm_stats) >= observed)

    return observed, p_value

# If p-value < 0.05, missingness depends on cause category
permutation_test(df, 'CAUSE.CATEGORY', 'CUSTOMERS_MISSING')

```

Out[11]: (0.806005855888024, 0.0)

In [12]: *# Example Where It Likely Does NOT Depend*

```

# CUSTOMERS.AFFECTED missingness does NOT depend on ANOMALY.LEVEL -- p
permutation_test(df, 'ANOMALY.LEVEL', 'CUSTOMERS_MISSING')

```

Out[12]: (0.5384615384615384, 0.502)

In [13]: *# Run permutation test and collect stats for plotting*

```

observed, p_val = permutation_test(df, 'CAUSE.CATEGORY', 'CUSTOMERS_MI

# Re-run to collect perm stats for plot
perm_stats = []
for _ in range(1000):

```

```
shuffled = df.copy()
shuffled['CUSTOMERS_MISSING'] = np.random.permutation(shuffled['CU
stat = (
    shuffled.groupby('CAUSE.CATEGORY')['CUSTOMERS_MISSING'].mean()
    shuffled.groupby('CAUSE.CATEGORY')['CUSTOMERS_MISSING'].mean()
)
perm_stats.append(stat)

fig = px.histogram(
    x=perm_stats,
    title='Empirical Distribution of Missingness Test Statistic (Cause
    labels={'x': 'Difference in Missingness Rates'})
)
fig.add_vline(x=observed, line_dash='dash', line_color='red',
              annotation_text=f'Observed: {round(observed, 3)}')
fig.update_layout(plot_bgcolor='white', title=dict(x=0.5))
fig.write_html('../figures/missingness/cause_missingness_dist.html', i
fig.show()
```

Step 4: Hypothesis Testing

Null Hypothesis (H_0): The mean outage duration during abnormal climate conditions (warm or cold) is equal to the mean outage duration during normal conditions.

- $\mu_{\text{abnormal}} = \mu_{\text{normal}}$

Alternative Hypothesis (H_1): The mean outage duration during abnormal climate conditions is greater than the mean outage duration during normal conditions.

- $\mu_{\text{abnormal}} > \mu_{\text{normal}}$

Test Statistic The difference in mean outage duration (in hours) between abnormal and normal climate conditions.

- Mean duration (abnormal) – Mean duration (normal)

```
In [14]: def climate_duration_permutation_test(df, n_perm=5000):

    # Bin warm and cold into a single 'abnormal' group
    climate_group = df['CLIMATE.CATEGORY'].replace({
        'warm': 'abnormal',
        'cold': 'abnormal',
        'normal': 'normal'
    })

    duration_hours = df['OUTAGE.DURATION'] / 60

    # Observed statistic: mean duration (abnormal) – mean duration (normal)
    observed = (
        duration_hours[climate_group == 'abnormal'].mean()
        -
        duration_hours[climate_group == 'normal'].mean()
    )

    perm_stats = []

    for _ in range(n_perm):
        shuffled = np.random.permutation(climate_group.values)
        stat = (
            duration_hours[shuffled == 'abnormal'].mean()
            -
            duration_hours[shuffled == 'normal'].mean()
        )

        perm_stats.append(stat)

    perm_stats = np.array(perm_stats)

    # One-sided p-value: proportion of permutations >= observed
```



```
p_value = np.mean(perm_stats >= observed)

return observed, p_value
```

```
In [15]: observed_stat, p_value = climate_duration_permutation_test(df)
observed_stat, p_value
```

```
Out[15]: (3.491106864406433, 0.2666)
```

```
In [16]: # Plot empirical distribution of permutation test statistic
fig = px.histogram(
    x=perm_stats,
    title='Empirical Distribution of Permutation Test Statistic',
    labels={'x': 'Difference in Mean Duration (hours)'}
)
fig.add_vline(x=observed_stat, line_dash='dash', line_color='red',
              annotation_text=f'Observed: {round(observed_stat, 3)}')
fig.update_layout(plot_bgcolor='white', title=dict(x=0.5))
fig.write_html('../figures/hypothesis/climate_permutation_dist.html',
fig.show()
```

Results

- Observed difference = 3.49 hours
- p-value = 0.267
- Fail to reject H_0

Conclusion

Because the p-value (0.267) is greater than 0.05, we fail to reject the null hypothesis.

Although outages during abnormal climate conditions lasted approximately 3.49 hours longer on average, this difference is not statistically significant. The observed difference could reasonably have occurred due to random variation.

Therefore, we do not find sufficient evidence to conclude that abnormal climate conditions lead to longer outage durations.

Step 5: Framing a Prediction Problem

Prediction Problem

Can we predict, using only information available at the time an outage begins, whether the outage will become severe (defined as lasting 12 or more hours)?

This is a **binary classification** problem. The response variable is `SEVERE_OUTAGE`, a binary label where Severe indicates an outage lasting 12 or more hours and Not Severe indicates otherwise. If a utility could flag a severe outage in its first moments, they could pre-position crews, notify customers earlier, and allocate resources before the situation escalates, potentially saving significant cost and reducing customer impact.

To ensure the model is realistic, only variables known at outage onset are used as predictors. Information that becomes available after the outage progresses (such as restoration time, total duration, customers affected, and demand loss) is excluded to prevent data leakage.

Evaluation Metric

The dataset is roughly balanced between severe and not severe outages. Even so, accuracy alone is a poor metric here because missing a severe outage (false

negative) is far more costly than over-preparing for one that turns out to be minor (false positive). We use **F1-score** to balance precision and recall, and **PR AUC (Average Precision)** as a summary metric that captures model performance across all classification thresholds.

Step 6: Baseline Model

```
In [17]: # Filter data for baseline model
def data_baseline(df, severe_hours=12):

    df = df.copy()

    # Create target
    df['SEVERE_OUTAGE'] = (df['OUTAGE.DURATION'] / 60 >= severe_hours)

    # Map Target Names
    df['SEVERE_OUTAGE'] = df['SEVERE_OUTAGE'].map({1: 'Severe', 0: 'No'})

    # Use already-created timestamp
    df['START_HOUR'] = df['OUTAGE.START'].dt.hour
    df['START_DAYOFWEEK'] = df['OUTAGE.START'].dt.dayofweek
    df['START_QUARTER'] = df['OUTAGE.START'].dt.quarter

    feature_cols = [
        'YEAR', 'MONTH', 'START_HOUR', 'START_DAYOFWEEK', 'START_QUARTER',
        'U.S._STATE', 'POSTAL.CODE', 'NERC.REGION', 'CLIMATE.REGION',
        'CLIMATE.CATEGORY', 'ANOMALY.LEVEL',
        'CAUSE.CATEGORY', 'CAUSE.CATEGORY.DETAIL',
        'TOTAL.CUSTOMERS', 'POPDEN_URBAN', 'PC.REALGSP.STATE', 'UTIL.CATEGORY'
    ]

    # Drop leakage columns
    df = df.drop(columns=[
        'OUTAGE.RESTORATION',
        'OUTAGE.DURATION',
        'CUSTOMERS.AFFECTED',
        'DEMAND.LOSS.MW'
    ], errors='ignore')

    baseline_df = df[feature_cols + ['SEVERE_OUTAGE', 'OUTAGE.START']]

    return baseline_df
```

```
In [18]: # Baseline training pipeline
def train_baseline(baseline_df, cutoff_date):
    # Temporal split: train on outages before cutoff, test on outages after cutoff
    train_df = baseline_df[baseline_df['OUTAGE.START'] < cutoff_date]
    test_df = baseline_df[baseline_df['OUTAGE.START'] >= cutoff_date]
```

```

y_train = train_df['SEVERE_OUTAGE']
y_test = test_df['SEVERE_OUTAGE']
X_train = train_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
X_test = test_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])

cat_cols = X_train.select_dtypes(include=['object']).columns.tolist()
num_cols = X_train.select_dtypes(exclude=['object']).columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
        ]), num_cols),
        ('cat', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore')),
        ]), cat_cols),
    ]
)

baseline_model = Pipeline(steps=[
    ('prep', preprocessor),
    ('clf', LogisticRegression(max_iter=2000, class_weight='balanced'))
])

baseline_model.fit(X_train, y_train)
preds = baseline_model.predict(X_test)
proba = baseline_model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, preds, digits=3))
print("Average Precision (PR AUC):", round(average_precision_score(y_test, proba), 3))

return baseline_model

```

```

In [19]: baseline_df = data_baseline(df, severe_hours=12)
baseline_model = train_baseline(baseline_df, cutoff_date="2015-01-01")

with open('../models/baseline_model.pkl', 'wb') as f:
    pickle.dump(baseline_model, f)
print("\nBaseline model saved to baseline_model.pkl")

```

	precision	recall	f1-score	support
Not Severe	0.764	0.802	0.783	101
Severe	0.565	0.510	0.536	51
accuracy			0.704	152
macro avg	0.665	0.656	0.659	152
weighted avg	0.697	0.704	0.700	152

Average Precision (PR AUC): 0.6014

Baseline model saved to baseline_model.pkl

Baseline Model Summary

The baseline model is a logistic regression classifier trained on features available at outage onset.

Features used (17 total):

- **Quantitative (10):** YEAR , MONTH , ANOMALY.LEVEL , TOTAL.CUSTOMERS , POPDEN_URBAN , PC.REALGSP.STATE , UTIL.CONTRI , START_HOUR , START_DAYOFWEEK , START_QUARTER
- **Nominal (7):** CAUSE.CATEGORY , CAUSE.CATEGORY.DETAIL , U.S._STATE , POSTAL.CODE , NERC.REGION , CLIMATE.REGION , CLIMATE.CATEGORY

Nominal features were encoded using `OneHotEncoder` . Missing values in numeric columns were imputed with the median and missing categorical values with the most frequent value. All steps are implemented in a single `sklearn` Pipeline.

Performance: The model was trained on outages before 2015 and tested on outages from 2015 onward. On the test set it achieved 70.4% accuracy, an F1-score of 0.536 on the severe class, and a PR AUC of 0.60.

Is this a good model? Not yet. While 70% accuracy sounds reasonable, the F1 of 0.54 on the severe class, the class that actually matters, shows the model struggles to reliably identify outages that will become major events. This is the baseline to beat in the final model.

Step 7: Final Model

Feature Engineering

New features were engineered from existing columns to improve predictive power:

IS_WEEKEND - derived from `START_DAYOFWEEK` . Utility crews are harder to mobilize on weekends, which could affect how quickly an outage is classified and resolved.

IS_SUMMER - derived from `MONTH` . Summer months (June, July, August) represent peak demand season when the grid is under more stress, making

severe outages more likely.

LOG_TOTAL_CUSTOMERS - log transformation of **TOTAL.CUSTOMERS**. The raw customer count is heavily right-skewed. The log transformation compresses the scale and captures the diminishing relationship between state size and outage severity.

STATE_SEVERE_RATE - historical rate of severe outages per state, computed from training data only to avoid leakage. States with a history of severe outages likely have infrastructure or geographic characteristics that make them prone to long outages, making this a strong predictive signal.

CAUSE_CLIMATE - combining **CAUSE.CATEGORY** and **CLIMATE.CATEGORY**. A severe weather outage during a cold climate period may behave differently than the same cause during normal conditions, and this feature allows the model to capture those interaction effects.

```
In [20]: # Filter data for final model
def data_final(df, cutoff_date="2015-01-01", severe_hours=12):
    final_df = df.copy()

    # Create target
    final_df['SEVERE_OUTAGE'] = (final_df['OUTAGE.DURATION'] / 60 >= s
    final_df['SEVERE_OUTAGE'] = final_df['SEVERE_OUTAGE'].map({True: 'Severe', False: 'Not Severe'})

    # Time features
    final_df['START_HOUR'] = final_df['OUTAGE.START'].dt.hour
    final_df['START_DAYOFWEEK'] = final_df['OUTAGE.START'].dt.dayofweek
    final_df['START_QUARTER'] = final_df['OUTAGE.START'].dt.quarter
    final_df['IS_WEEKEND'] = final_df['START_DAYOFWEEK'].isin([5, 6]).astype(int)
    final_df['IS_SUMMER'] = final_df['MONTH'].isin([6, 7, 8]).astype(int)
    final_df['LOG_TOTAL_CUSTOMERS'] = np.log1p(final_df['TOTAL.CUSTOMERS'])
    final_df['CAUSE_CLIMATE'] = final_df['CAUSE.CATEGORY'] + '_' + final_df['CLIMATE.CATEGORY']

    # Compute state severe rate from training data only to avoid leakage
    train_df = final_df[final_df['OUTAGE.START'] < cutoff_date]
    state_severe_rate = (
        train_df.groupby('U.S._STATE')['SEVERE_OUTAGE']
        .apply(lambda x: (x == 'Severe').mean())
        .rename('STATE_SEVERE_RATE')
    )
    final_df = final_df.merge(state_severe_rate, on='U.S._STATE', how='left')
    # Fill states that only appear in test with overall training severe rate
    overall_rate = (train_df['SEVERE_OUTAGE'] == 'Severe').mean()
    final_df['STATE_SEVERE_RATE'] = final_df['STATE_SEVERE_RATE'].fillna(overall_rate)

    feature_cols = [
        'YEAR', 'MONTH', 'START_HOUR', 'START_DAYOFWEEK', 'START_QUARTER',
        'IS_WEEKEND', 'IS_SUMMER', 'LOG_TOTAL_CUSTOMERS', 'STATE_SEVERE_RATE']
```

```

        'U.S._STATE', 'NERC.REGION', 'CLIMATE.REGION',
        'CLIMATE.CATEGORY', 'ANOMALY.LEVEL',
        'CAUSE.CATEGORY', 'CAUSE.CATEGORY.DETAIL',
        'POPDEN_URBAN', 'PC.REALGSP.STATE', 'UTIL.CONTRI'
    ]

    final_df = final_df.drop(columns=[
        'OUTAGE.RESTORATION', 'OUTAGE.DURATION',
        'CUSTOMERS.AFFECTED', 'DEMAND.LOSS.MW', 'POSTAL.CODE'
    ], errors='ignore')

    return final_df[feature_cols + ['SEVERE_OUTAGE', 'OUTAGE.START']]

final_df = data_final(df, cutoff_date="2015-01-01")

```

```

In [21]: # Final training pipeline
def train_final(final_df, cutoff_date, model, model_name):
    # Temporal split: same cutoff as baseline for fair comparison
    train_df = final_df[final_df['OUTAGE.START'] < cutoff_date].copy()
    test_df = final_df[final_df['OUTAGE.START'] >= cutoff_date].copy()

    y_train = train_df['SEVERE_OUTAGE']
    y_test = test_df['SEVERE_OUTAGE']
    X_train = train_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
    X_test = test_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])

    cat_cols = X_train.select_dtypes(include=['object', 'str']).column
    num_cols = X_train.select_dtypes(exclude=['object', 'str']).column

    preprocessor = ColumnTransformer(transformers=[
        ('num', SimpleImputer(strategy='median'), num_cols),
        ('cat', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore'))
        ]), cat_cols)
    ])

    pipeline = Pipeline(steps=[
        ('prep', preprocessor),
        ('clf', model)
    ])

    pipeline.fit(X_train, y_train)
    preds = pipeline.predict(X_test)
    # Index into predict_proba by class name to handle any label order
    proba = pipeline.predict_proba(X_test)[ :, list(pipeline.classes_)

    print(f"\n=== {model_name} ===")
    print(classification_report(y_test, preds, digits=3))
    print("PR AUC:", round(average_precision_score(y_test, proba, pos_
    return pipeline

```

```
In [22]: # Compare three classifiers with balanced class weights to handle clas
final_df = data_final(df)
models = {
    'Logistic Regression': LogisticRegression(max_iter=2000, class_w
    'Random Forest': RandomForestClassifier(class_weight='balanced', r
    'LightGBM': LGBMClassifier(random_state=2, verbose=-1, class_weigh
}
print("Results: Weighted Models\n")
results = {}
for name, model in models.items():
    results[name] = train_final(final_df, cutoff_date="2015-01-01", mo
```

Results: Weighted Models

=== Logistic Regression ===

	precision	recall	f1-score	support
Not Severe	0.824	0.832	0.828	101
Severe	0.660	0.647	0.653	51
accuracy			0.770	152
macro avg	0.742	0.739	0.741	152
weighted avg	0.769	0.770	0.769	152

PR AUC: 0.6642

=== Random Forest ===

	precision	recall	f1-score	support
Not Severe	0.842	0.842	0.842	101
Severe	0.686	0.686	0.686	51
accuracy			0.789	152
macro avg	0.764	0.764	0.764	152
weighted avg	0.789	0.789	0.789	152

PR AUC: 0.6238

=== LightGBM ===

	precision	recall	f1-score	support
Not Severe	0.804	0.851	0.827	101
Severe	0.667	0.588	0.625	51
accuracy			0.763	152
macro avg	0.735	0.720	0.726	152
weighted avg	0.758	0.763	0.759	152

PR AUC: 0.6605

```
In [23]: # Compare same three classifiers without class weighting to see if it
models_no_weights = {
```



```

'Logistic Regression': LogisticRegression(max_iter=2000),
'Random Forest': RandomForestClassifier(random_state=2),
'LightGBM': LGBMClassifier(random_state=2, verbose=-1)
}
print("Results: Non-Weighted Models\n")
results = {}
for name, model in models_no_weights.items():
    results[name] = train_final(final_df, cutoff_date="2015-01-01", mo

```

Results: Non-Weighted Models

=== Logistic Regression ===

	precision	recall	f1-score	support
Not Severe	0.845	0.812	0.828	101
Severe	0.655	0.706	0.679	51
accuracy			0.776	152
macro avg	0.750	0.759	0.754	152
weighted avg	0.781	0.776	0.778	152

PR AUC: 0.6547

=== Random Forest ===

	precision	recall	f1-score	support
Not Severe	0.827	0.851	0.839	101
Severe	0.688	0.647	0.667	51
accuracy			0.783	152
macro avg	0.757	0.749	0.753	152
weighted avg	0.780	0.783	0.781	152

PR AUC: 0.632

=== LightGBM ===

	precision	recall	f1-score	support
Not Severe	0.832	0.832	0.832	101
Severe	0.667	0.667	0.667	51
accuracy			0.776	152
macro avg	0.749	0.749	0.749	152
weighted avg	0.776	0.776	0.776	152

PR AUC: 0.6804

Hyperparameter Tuning

I used `GridSearchCV` with 3-fold cross validation to search over the following LightGBM hyperparameters:

n_estimators - controls the number of boosting rounds (trees). More trees can improve performance but risk overfitting on small datasets. Testing a range from 10 to 100 to find the right balance.

learning_rate - controls how much each tree contributes to the final prediction. Lower rates are more conservative and generally need more trees to compensate. Testing from 0.005 to 0.1.

num_leaves - controls tree complexity in LightGBM. Unlike max_depth, num_leaves directly caps how many leaf nodes each tree can have. Smaller values reduce overfitting. Testing from 2 to 50.

min_child_samples - minimum number of data points required in a leaf node. Higher values act as regularization, preventing the model from fitting to very small subsets of the data. Important for a dataset of this size. Testing from 3 to 50.

subsample - fraction of training rows used per tree. Adding randomness through subsampling helps generalization. Testing 0.6, 0.8, and 1.0.

3-fold cross validation was chosen over the standard 5-fold because the training set contains approximately 1,200 rows. With 5 folds, each fold would train on only ~960 rows, which is too small to reliably estimate model performance. 3 folds keeps each training fold at ~800 rows, providing more stable estimates.

```
In [24]: # Tune lightgbm
def tune_lightgbm(final_df, cutoff_date):
    train_df = final_df[final_df['OUTAGE.START'] < cutoff_date].copy()
    test_df = final_df[final_df['OUTAGE.START'] >= cutoff_date].copy()
    y_train = train_df['SEVERE_OUTAGE']
    y_test = test_df['SEVERE_OUTAGE']
    X_train = train_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
    X_test = test_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
    cat_cols = X_train.select_dtypes(include=['object', 'str']).columns
    num_cols = X_train.select_dtypes(exclude=['object', 'str']).columns
    preprocessor = ColumnTransformer(transformers=[
        ('num', SimpleImputer(strategy='median'), num_cols),
        ('cat', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore'))
        ]), cat_cols)
    ])
    pipeline = Pipeline(steps=[
        ('prep', preprocessor),
        ('clf', LGBMClassifier(random_state=2, verbose=-1))
    ])
    param_grid = {
```

```

        'clf__n_estimators': [10, 20, 50, 100],
        'clf__learning_rate': [0.005, 0.01, 0.05, 0.1],
        'clf__num_leaves': [2, 5, 10, 20, 31, 50],
        'clf__min_child_samples': [3, 5, 10, 20, 30, 50],
        'clf__subsample': [0.6, 0.8, 1.0],
    }
    # Optimize for F1 on the severe class
    scorer = make_scorer(f1_score, pos_label='Severe')
    # 3-fold CV chosen over 5-fold to keep each training fold large en
    grid_search = GridSearchCV(
        pipeline,
        param_grid,
        scoring=scorer,
        cv=3,
        n_jobs=-1,
        verbose=1
    )
    grid_search.fit(X_train, y_train)
    print("Best params:", grid_search.best_params_)
    print("Best CV F1:", round(grid_search.best_score_, 4))
    best_model = grid_search.best_estimator_
    preds = best_model.predict(X_test)
    proba = best_model.predict_proba(X_test)[:, list(best_model.class_
    print("\n=== Tuned LightGBM ===")
    print(classification_report(y_test, preds, digits=3))
    print("PR AUC:", round(average_precision_score(y_test, proba, pos_
    return best_model, grid_search

```

```
best_lgbm, lgbm_grid = tune_lightgbm(final_df, cutoff_date="2015-01-01
```

Fitting 3 folds for each of 1728 candidates, totalling 5184 fits
 Best params: {'clf__learning_rate': 0.01, 'clf__min_child_samples': 10,
 'clf__n_estimators': 50, 'clf__num_leaves': 5, 'clf__subsample': 0.6}
 Best CV F1: 0.7757

```

=== Tuned LightGBM ===
              precision    recall  f1-score   support

   Not Severe       0.875      0.832      0.853        101
      Severe       0.696      0.765      0.729         51

 accuracy                   0.809        152
  macro avg       0.786      0.798      0.791        152
 weighted avg       0.815      0.809      0.811        152

```

PR AUC: 0.6466

```

In [25]: # Train final LightGBM on full dataset using best hyperparameters from
def train_final_model(final_df):
    X = final_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
    y = final_df['SEVERE_OUTAGE']
    cat_cols = X.select_dtypes(include=['object', 'str']).columns.tolist()
    num_cols = X.select_dtypes(exclude=['object', 'str']).columns.tolist()

```

```

preprocessor = ColumnTransformer(transformers=[
    ('num', SimpleImputer(strategy='median'), num_cols),
    ('cat', Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ]), cat_cols)
])
# Best hyperparameters from GridSearchCV
final_model = Pipeline(steps=[
    ('prep', preprocessor),
    ('clf', LGBMClassifier(
        random_state=2,
        verbose=-1,
        learning_rate=0.01,
        min_child_samples=10,
        n_estimators=50,
        num_leaves=5,
        subsample=0.6
    ))
])
final_model.fit(X, y)
with open('../models/final_model.pkl', 'wb') as f:
    pickle.dump(final_model, f)
print("Final model saved to final_model.pkl")
return final_model

final_model = train_final_model(final_df)

```

Final model saved to final_model.pkl

```

In [26]: # Get predictions on test set
test_df = final_df[final_df['OUTAGE.START'] >= '2015-01-01'].copy()
X_test = test_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
y_test = test_df['SEVERE_OUTAGE']

preds = final_model.predict(X_test)

cm = confusion_matrix(y_test, preds, labels=['Not Severe', 'Severe'])

fig = ff.create_annotated_heatmap(
    z=cm,
    x=['Predicted: Not Severe', 'Predicted: Severe'],
    y=['Actual: Not Severe', 'Actual: Severe'],
    colorscale='Blues',
    showscale=True
)

fig.update_layout(
    title='Confusion Matrix - Final LightGBM Model',
    title_x=0.5
)

fig.write_html('../figures/final_model/confusion_matrix.html', include

```

```
fig.show()
```

Final Model

The final model is a **LightGBM classifier** tuned via 3-fold GridSearchCV. The best hyperparameters were:

- `learning_rate` : 0.01
- `min_child_samples` : 10
- `n_estimators` : 50
- `num_leaves` : 5
- `subsample` : 0.6

Comparison to Baseline:

Metric	Baseline (Log Reg)	Final (LightGBM)
Accuracy	70.4%	80.9%

F1 (Severe)	0.536	0.729
PR AUC	0.60	0.6466

The final model meaningfully improved across all metrics, with the most significant gain on the severe class F1 score, which is the metric that matters most for real utility decision-making.

Step 8: Fairness Analysis

We evaluate whether our final model performs equally well for outages occurring in high-density urban states versus lower-density suburban and rural states. This is an important equity question -- if the model systematically underperforms for less-populated areas, utilities in those regions may receive less reliable early warning about severe outages, despite often having fewer resources to respond.

We define our two groups using `POPDEN_URBAN`, the population density of urban areas in the state where the outage occurred. Following the U.S. Census Bureau's finding that the average population density inside U.S. cities is approximately 1,600 people per square mile, we use this as our threshold:

- **Group X (Low Density):** States with `POPDEN_URBAN` < 1,600 -- suburban and rural
- **Group Y (High Density):** States with `POPDEN_URBAN` >= 1,600 -- urban

Evaluation Metric: F1 score on the Severe class -- the class that matters most for utility response planning.

Null Hypothesis: The model is fair. Its F1 score for low density and high density states are roughly the same, and any differences are due to random chance.

Alternative Hypothesis: The model is unfair. Its F1 score is lower for low density states than for high density states.

Test Statistic: Difference in F1 score (high density minus low density)

Significance Level: 0.05

```
In [27]: def fairness_permutation_test(model, final_df, cutoff_date, n_perm=100
        test_df = final_df[final_df['OUTAGE.START'] >= cutoff_date].copy()

        X_test = test_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
        y_test = test_df['SEVERE_OUTAGE']

        preds = model.predict(X_test)
```

```

# Create density groups on test set
high_density = test_df['POPDEN_URBAN'] >= 1600
low_density = test_df['POPDEN_URBAN'] < 1600

# Observed test statistic
f1_high = f1_score(y_test[high_density], preds[high_density], pos_
f1_low = f1_score(y_test[low_density], preds[low_density], pos_lab
observed_stat = f1_high - f1_low

print(f"F1 High Density: {round(f1_high, 4)}")
print(f"F1 Low Density: {round(f1_low, 4)}")
print(f"Observed Difference: {round(observed_stat, 4)}")

# Permutation test
perm_stats = []
for _ in range(n_perm):
    shuffled_groups = np.random.permutation(high_density.values)
    f1_h = f1_score(y_test[shuffled_groups], preds[shuffled_groups]
    f1_l = f1_score(y_test[~shuffled_groups], preds[~shuffled_grou
    perm_stats.append(f1_h - f1_l)

perm_stats = np.array(perm_stats)
p_value = np.mean(perm_stats >= observed_stat)

print(f"P-value: {round(p_value, 4)}")

# Plot empirical distribution
fig = px.histogram(
    x=perm_stats,
    title='Empirical Distribution of F1 Difference (High - Low Den
    labels={'x': 'F1 Score Difference'}
)
fig.add_vline(x=observed_stat, line_dash='dash', line_color='red',
              annotation_text=f'Observed: {round(observed_stat, 4)}
fig.update_layout(plot_bgcolor='white', title=dict(x=0.5))
fig.write_html('../figures/fairness/fairness_permutation_dist.html
fig.show()

return p_value

p_value = fairness_permutation_test(final_model, final_df, cutoff_date

```

F1 High Density: 0.7191
 F1 Low Density: 0.8571
 Observed Difference: -0.138
 P-value: 0.849

Results

Metric	Value
F1 Score (High Density)	0.7191
F1 Score (Low Density)	0.8571
Observed Difference	-0.138
P-value	0.849
Significance Level	0.05

Conclusion: I fail to reject the null hypothesis ($p = 0.846$). The observed difference of -0.138 in F1 score between high and low density states is not statistically significant, as differences of this magnitude occur frequently by random chance alone. There is not sufficient evidence to conclude that the final LightGBM model performs unfairly across urban density groups.

Interestingly, the model actually performs slightly better for low density states ($F1 = 0.857$) than high density states ($F1 = 0.719$), though this difference is not meaningful given the small test set size.

In []: